

Netty初识（下）

开发客户端示例程序

北京理工大学计算机学院
金旭亮



概述



前一讲介绍了服务端示例程序的代码，这一讲介绍客户端程序的代码。

在本讲示例中，客户端要完成的工作相当简单，就是尝试连接服务器，连接成功之后，输出一条信息，然后立即退出。

编写客户端处理程序



与服务端一样，客户端程序也在ChannelHandler中编写相应逻辑：

```
@ChannelHandler.Sharable
public class MyClientHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        super.channelActive(ctx);
        System.out.println("MyClientHandler channelActive");
        //现在可以向Server发送数据.....
        //立即断开连接
        ctx.close();
    }
}
```



当客户端连接服务器成功之后，上述代码中的channelActive()方法会被调用，示例输出相应信息之后，调用ChannelHandlerContext方法，立即断开连接。

客户端应用启动流程



```
public class MyNettyClient {  
    public static void main(String[] args) {  
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();
```

```
        Bootstrap bootstrap = new Bootstrap();  
        bootstrap.group(workerGroup)  
            .channel(NioSocketChannel.class)  
            .handler(new MyClientHandler());
```

Bootstrap对象有一个connect方法
用于连接服务器

//连接服务器

```
        var resultFuture = bootstrap.connect("127.0.0.1", 9000).addListener(future -> {  
            if (future.isSuccess()) {  
                System.out.println("连接成功。");  
            } else {  
                System.out.println("连接失败。");  
            }  
        });
```

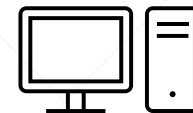
connect方法返回ChannelFuture，是一个
异步操作，可以挂接一个监听器

//通道关闭后，关闭线程池

```
        var closeFuture = resultFuture.channel().closeFuture().addListener(future -> {  
            workerGroup.shutdownGracefully();  
            System.out.println("程序已经退出。");  
        });
```

```
    }  
}
```

运行截图



先运行服务端程序监听9000端口，之后启动客户端，可以看到，成功连接之后，客户端会调用MyClientHandler中定义的channelActive方法，之后立即退出。

```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
9000端口绑定成功。
有客户端连接: /127.0.0.1:12803
```

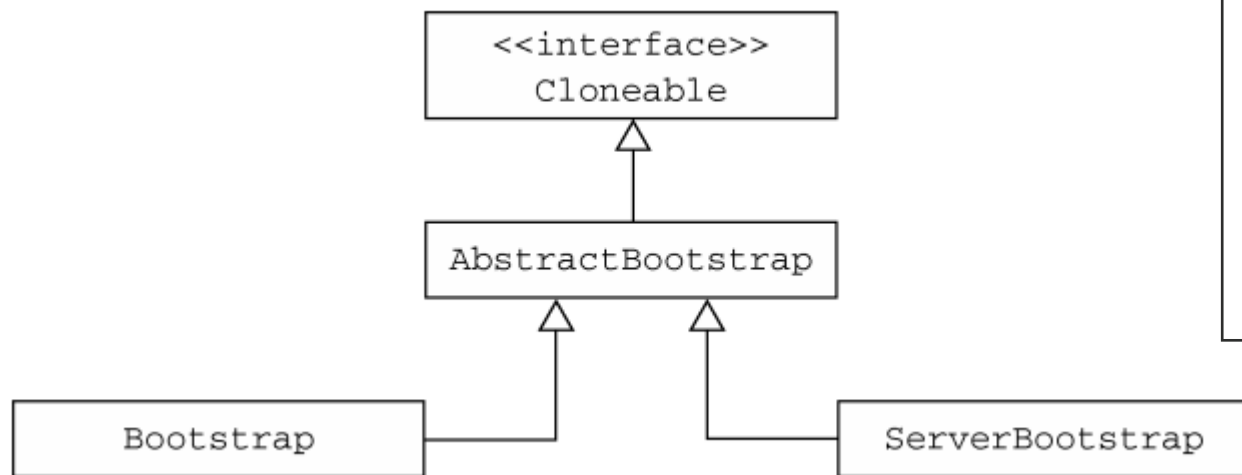
```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功。
MyClientHandler channelActive
程序已经退出。

Process finished with exit code 0
```

当有客户端连接请求进来，服务端会调用MyServerHandler中定义的channelActive方法，输出客户端的地址及所使用端口信息。

由于没有写代码关闭服务端通道，所以，服务端程序会继续运行，等待下一个连接.....

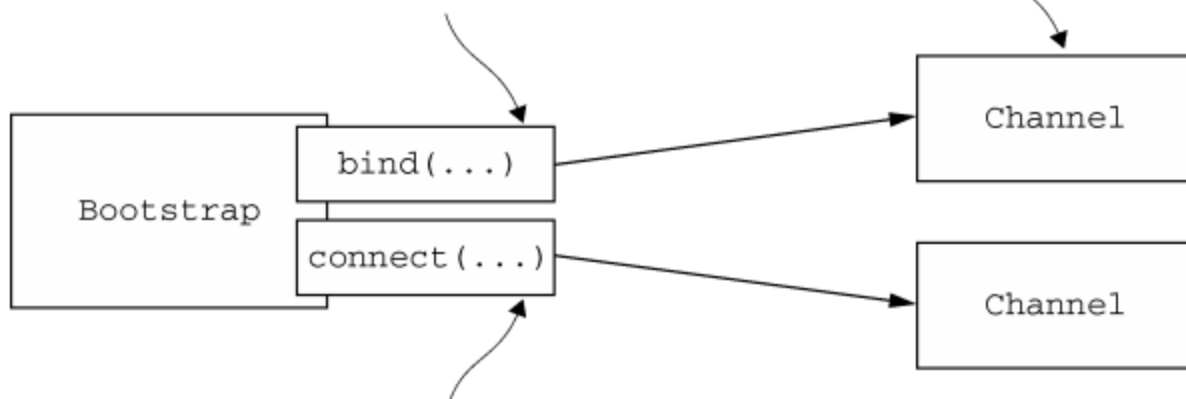
Netty应用的启动过程



Netty应用启动时，需要完成一系列的初始化工作，这些功能，是由引导类实例负责的，分为Bootstrap（客户端）和ServerBootstrap（服务端）两类。

客户端引导过程

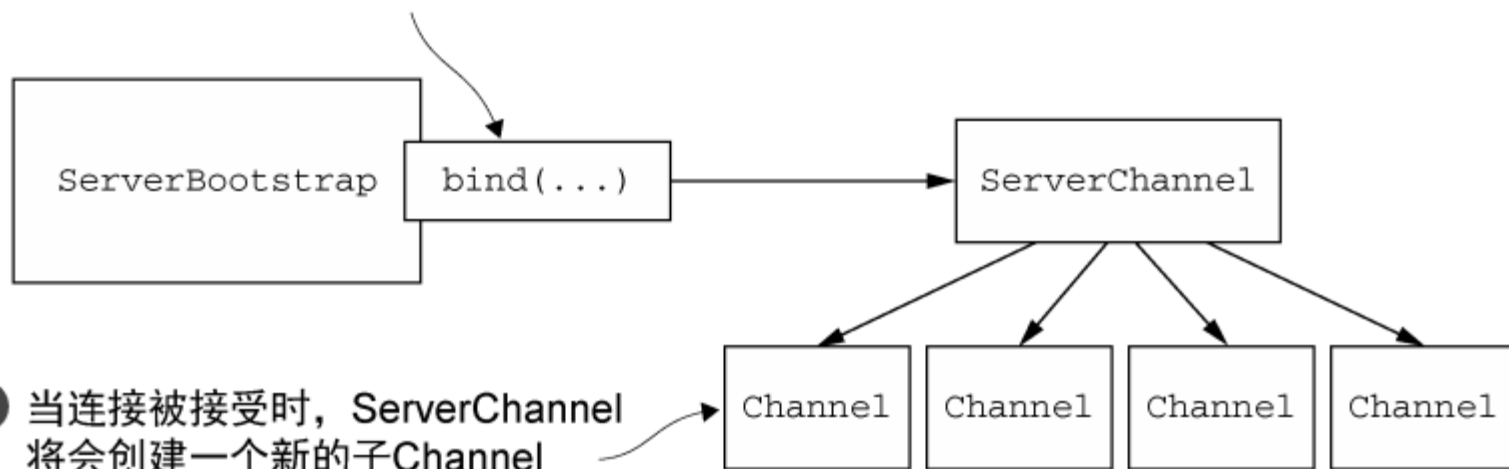
- 1 Bootstrap类将会在bind()方法被调用后创建一个新的Channel，在这之后将会调用connect()方法以建立连接



- 2 在connect()方法被调用后，Bootstrap类将会创建一个新的Channel

Server端启动过程

- 1 当bind()方法被调用时，将会创建一个ServerChannel



- 2 当连接被接受时，ServerChannel将会创建一个新的子Channel

小结：理解Netty各组件的职责



Netty组件

Channel

EventLoopGroup

ChannelFuture

JDK标准组件

Socket

线程池

CompletableFuture

Netty中的许多组件，往往都是基于JDK中原有组件演化而来的，因此，通过对比两者，有助于同学们理解其职责。